

The Impact of Dormant Defects on Defect Prediction: A Study of 19 Apache Projects

DAVIDE FALESSI, University of Rome Tor Vergata, Italy
AALOK AHLUWALIA, California Polytechnic State University, USA
MASSIMILIANO DI PENTA, University of Sannio, Italy

Defect prediction models can be beneficial to prioritize testing, analysis, or code review activities, and has been the subject of a substantial effort in academia, and some applications in industrial contexts. A necessary precondition when creating a defect prediction model is the availability of defect data from the history of projects. If this data is noisy, the resulting defect prediction model could result to be unreliable. One of the causes of noise for defect datasets is the presence of "dormant defects," i.e., of defects discovered several releases after their introduction. This can cause a class to be labeled as defect-free while it is not, and is, therefore "snoring." In this article, we investigate the impact of snoring on classifiers' accuracy and the effectiveness of a possible countermeasure, i.e., dropping too recent data from a training set. We analyze the accuracy of 15 machine learning defect prediction classifiers, on data from more than 4,000 defects and 600 releases of 19 open source projects from the Apache ecosystem. Our results show that on average across projects (i) the presence of dormant defects decreases the recall of defect prediction classifiers, and (ii) removing from the training set the classes that in the last release are labeled as not defective significantly improves the accuracy of the classifiers. In summary, this article provides insights on how to create defects datasets by mitigating the negative effect of dormant defects on defect prediction.

CCS Concepts: • **Software and its engineering → Software defect analysis;**

Additional Key Words and Phrases: Defect prediction, fix-inducing changes, dataset bias

ACM Reference format:
Davide Falessi, Aalok Ahluwalia, and Massimiliano Di Penta. 2021. The Impact of Dormant Defects on Defect Prediction: A Study of 19 Apache Projects. *ACM Trans. Softw. Eng. Methodol.* 31, 1, Article 4 (September 2021), 26 pages.
https://doi.org/10.1145/3467895

1 INTRODUCTION

Defect prediction models aim at identifying software artifacts that are likely to exhibit a defect [47, 54–56, 72, 75]. The main purpose of defect prediction is to reduce the cost of testing, analysis, or code review, by prioritizing developers' effort on specific artifacts. In this area, researchers have

Authors' addresses: D. Falessi, University of Rome Tor Vergata, Italy, Viale del politecnico 1, 00133 Roma Italy; email: falessi@ing.uniroma2.it; A. Ahluwalia, California Polytechnic State University, 1 Grand Avenue San Luis Obispo, CA 93407 USA; email: ahluwali@calpoly.edu; M. Di Penta, University of Sannio, Piazza Guerrazzi 82100 Benevento BN Italy; email: dipenta@unisannio.it.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2021 Association for Computing Machinery.
1049-331X/2021/09-ART4 $15.00
https://doi.org/10.1145/3467895

proposed the use of different models to predict defects, leveraging, for example, product metrics [4, 24, 26, 34, 52], process metrics [48], the knowledge of where previous defects occurred [37, 56], information about change-inducing fixes [33, 35], and, recently, using deep learning techniques to automatically engineer features from source code elements [74].

Several researchers have worked on improving the performance of defect estimation models using techniques such as tuning [20, 23, 70], re-balancing [1, 5], or feature selection [71]. In order to promote the usage and improvement of prediction models, researchers have provided means to create [21, 81], collect [16], and select [22, 50, 63] defect datasets. Ultimately, the reliability of a prediction model depends on the quality of the dataset [40, 65]. Previous work has identified sources of noise in datasets, including defect misclassification [6, 27, 36, 60, 69] and defect origin, [62] and proposed solutions to deal with them.

In summary, while researchers are attempting to build better models for defect prediction, the quality of the datasets used to train such a model plays a paramount role in obtaining accurate and usable results. To predict which classes are defect-prone, defect prediction classifiers need to be trained on a dataset where classes in releases have been previously labeled as defective or not. The best way to identify when a defect has been introduced in a software project is to use a "realistic approach," i.e., using the affected version reported in Jira [79]. Another viable approach is the SZZ algorithm [38, 66]. SZZ exploits the versioning system annotation mechanism (e.g., *git blame*) to determine, for the source code lines that have been changed in a defect fix, when they have last been changed before such a fix. On the one hand, assigning defects onto releases based on Jira fully reflects the outcome of the bug triaging; however, such information, in existing datasets, not only is fairly incomplete, but, also, it may not be available immediately as soon as a bug is discovered. On the other hand, SZZ is particularly useful in complementing the realistic approach, i.e., SZZ can be used for defects where the affect version is not reported in Jira. At the same time, it is known to be subject to imprecisions [13].

Regardless of the specific approach to identify the origin of defects, defects need to be fixed before any approach can be used to label classes as defective or not. Specifically, a defect may be discovered several releases after its introduction; this phenomenon has been called "dormant defect" [8, 61]. Thus, if we observe today the status of a class in its current version, it can be labeled as not defective while it is actually defective. Thus, a class has a noise that we call "snoring" under two conditions: (i) it contains at least one dormant defect, and (ii) it is labeled while all contained defects are still dormant.

In a recent article [3], we found that most of the defects in a project slept for more than 20% of releases and that the proportion of defective classes that are not identified as defective is more than 25% even if we ignore 50% of releases. Now that we know that defects sleep and datasets snore, we want to know and inhibit their negative impact on defect prediction. Thus, in this study, we address the following research questions:

- **RQ1:** *Does snoring reduce defect prediction performance?* We conjecture that the presence of snoring classes can bias classifiers during training and hence can negatively impact the classifiers' performance. In this research question, we investigate the extent to which the snoring that exists in open source projects impacts the performance of classifiers in predicting defect-prone classes.
- **RQ2:** *Does defect prediction performance improve if we remove from the training set the classes that are labeled as non-defective in the last releases?* Snoring classes are, by definition, more frequent in more recent releases, i.e., in releases closer to when classes are labeled. Moreover, classes labeled as defective cannot snore. This happens because such recent releases contain defects that may not have been discovered yet. Therefore, we might remove noise and hence

improve the performance of the classifiers by ignoring in the training set the classes that in the last releases are labeled as not defective. At the same time, if we reduce the training set too much by removing several recent releases because they are potentially affected by snoring, the limited availability of data would also affect the classifiers' performance. In other words, is it better to train a classifier on a big noisy dataset or on a less noisy yet smaller dataset? Where is the sweet spot? In this research question, we measure the impact of a countermeasure for snoring, which consists of removing from the training set the classes that in the last releases are labeled as not defective.

In our empirical study, we analyze data from 19 Apache projects featuring a total of more than 4,000 defects and 600 releases. Also, we compare the performance of 15 different machine learning classifiers in the context of within-project across-release defect prediction. Our results show that

(1) Snoring decreases the performance of each of the 15 classifiers. More specifically, while precision is not significantly affected, the presence of snoring significantly affects recall, with a large effect size on each of the 15 classifiers. The effect size of snoring on other performance indicators, such as the **Matthews Correlation Coefficient** (**MCC**), the **Area Under the Receiving Operating Characteristics Curve** (**AUC**), and Cohen Kappa is medium or large.
(2) Removing from the training set the classes that are labeled as non-defective in the last release increases the classifiers' performance. For instance, Recall, F1, Kappa, and Matthews increase by about 30%. Removing a further release could further improve performance; however, going behind worsens the performances, because the classifiers are trained with less and outdated data.

The remainder of the article is structured as follows. Section 2 describes the empirical study design. Section 3 reports and discusses the results of the empirical investigation. Section 5 discusses the threats to the study validity. Section 6 discusses the related literature, focusing in particular on noise and imprecision on defect prediction, and on techniques to build defect prediction datasets. Finally, Section 7 concludes the article and outlines directions for future work.

2 EMPIRICAL STUDY DESIGN

The *goal* of this study is to investigate the effect of snoring data on classifiers' performance, their evaluation, and the effectiveness of a countermeasure. The *quality focus* is the performance of such models, which can be affected by snoring. The *perspective* of the study is of researchers or practitioners that want to improve the way defect prediction datasets are constructed so that classifiers can more accurately predict defect-prone classes.

2.1 Definition of Snoring Classes

To better understand when and how classes snore, let us consider a project with three releases, *r1*, *r2*, and *r3*. If a defect has been introduced in *r1* and only fixed in *r3*, then the presence of the defect would not be considered in *r1* and *r2*. If in *r1* and *r2* a class does not exhibit any other defect but the dormant defect in question, such a class will be erroneously labeled as non-defective in *r1* and *r2*. This class is a false negative and it is a snoring noise in defect dataset. If in *r1* and *r2* a class does exhibit at least another defect, then the defect sleeping in *r1* and *r2* does not impact the labeling of the class since this class is labeled as defective anyway. This class is a true positive. Table 1 reports a scenario of a project in which events related to defects (I = introduced, N = nothing, and F = fixed) happen in three releases (columns) and impact three classes (rows). A defect is defined as a post-release defect if it is fixed in a release after the one in which it has been introduced. Thus,

Table 1. An Example of a Project in Which Events (I = Introduced, N = Nothing, and F = Fixed) Happen in Three Releases and Three Different Classes

	r1	r2	r3
C1	IF	I	F
C2	I	N	F
C3	II	F	F

Table 2. Post-Release Defectiveness of a Class in a Specific Release

Dataset created at *r2*.	
Classes	r1
C1	ND
C2	ND
C3	D

Dataset created at *r3*.		
Classes	r1	r2
C1	ND	D
C2	D	D
C3	D	D

Table 3. How Dormant Defect Bias Datasets

Dataset created at *r2*.	
Classes	r1
C1	TN
C2	FN
C3	TP

Dataset created at *r3*.		
Classes	r1	r2
C1	TN	TP
C2	TP	TP
C3	TP	TP

a defective class is a class having at least one post-release defect. For instance, in column 2, row 2 of Table 1, a defect in class C1 is introduced and fixed in the same release, *r1*. Thus, C1 is not defective as it contains no post-release defect.

Based on the events described in Table 1, Table 2 reports the (post-release) defectiveness for a class, when it is labeled at *r2* (left-side), or *r3* (right-side). Each class in each release is labeled as **defective** (**D**) or **not defective** (**ND**). It is important to note that the label of C2 in *r1* changes on whether it is labeled at the end of *r2* or *r3*. Specifically, if C2 is labeled at the end of *r2*, then the introduction is not discovered and hence the class C2 at *r1* is labeled as not defective.

Table 3 reports the snoring caused by snoring, of a class in a release, according to when the dataset is created, in terms of FN = the class is erroneously marked as not defective despite being defective, TN = the class is marked as not defective and is not defective, and TP = the class is marked as defective. Specifically, C2 at *r1* is a FN for a dataset created at the end of *r2* (left-side table) and a TP for dataset created at *r3* (right-side table). Note that we consider FN and do not consider FP because the dormant defects cannot introduce FP, i.e., if a class is labeled as defective then one or more defects have been fixed already and no defect that is possibly discovered in the future can change the class label to non-defective. Finally, we note that the discussion above applies to all known labeling mechanisms such as SZZ [38, 66] and the realistic one [79]. The presence of snoring in the data is due to when the classes are labeled as defective or not, and it is entirely independent of the specific technique used to label classes such as the realistic or SZZ. As a matter of fact, both realistic and SZZ use the fixed tickets as input; thus, both approaches entirely fail in taking into consideration the bugs that are not fixed yet.

Table 4. Details of the Used Projects

Project	Releases	Days	Commits	Defects	AV in Jira
AVRO	46	3,528	1,770	114	0.60
BOOKKEEPER	22	2,880	2,056	184	0.32
CHUKWA	11	3,638	849	7	0.47
CONNECTORS	118	3,311	4,672	261	0.85
CRUNCH	17	2,663	1,055	132	0.48
FALCON	31	2,450	2,227	219	0.40
GIRAPH	10	2,719	1,096	123	0.29
IVY	17	4,977	2,973	133	0.89
OPENJPA	31	4,671	4,978	380	0.82
PROTON	51	2,513	3,929	53	0.69
SSHD	26	3,652	1,589	124	0.64
STORM	36	2,634	9,754	442	0.52
SYNCOPE	48	3,214	6,320	296	0.93
TAJO	13	2,407	2,273	286	0.23
TEZ	34	2,163	2,661	559	0.14
TOMEE	22	4,792	12,135	277	0.27
WHIRR	8	1,788	569	20	0.37
ZEPPELIN	14	2,067	4,048	200	0.37
ZOOKEEPER	49	3,923	1,820	219	0.71

The study *context* consists of data from 19 open source projects from the Apache ecosystem. We focused on Apache[1] projects rather than random GitHub projects because the former have a higher quality of defect annotation, and this also avoided us having to consider unrealistic/toy projects [49]. We select the 19 projects that are managed in Jira, versioned in Git, have at least 10 releases, have most of the commits related to Java code, and have the highest proportion of defects linked to commits in the source code. A defect is linked if it can be associated with at least one commit in the source code's commit log. Table 4 reports the details of the used 19 projects in terms of releases, days, commits, defects, and proportion of defects providing the affected version in Jira.

In the following, we describe the empirical design of each research question.

2.2 RQ1: Does Snoring Reduce Defect Prediction Performance?

We are in the shoes of a practitioner who uses previous project releases' data to predict which classes of the current release are defect-prone. Thus, we want to measure the performance of the same classifier when trained on a dataset with versus without snoring. In this research question, we test the null hypothesis: H_{01}: there is no difference between the prediction performance achieved by classifiers when trained on a dataset with snoring versus a dataset without snoring.

2.2.1 Independent Variables. The independent variable of this research question is the presence or absence of snoring in the training set.

2.2.2 Dependent Variables. The dependent variable is the **relative loss** (**RL**) that we measure as the relative distance between the performance of a classifier trained on a dataset with snoring versus trained on a dataset without snoring. Specifically,

[1] https://people.apache.org/phonebook.html.

$RL = |performance\ trained\ without\ snoring - performance\ trained\ with\ snoring|/$ $performance\ trained\ without\ snoring$.

Thus, the relative loss represents the detriment on performance caused by snoring, as observed by each different performance metric.

As performance metrics we used the following metrics:

— **True Positive (TP):** The class is actually defective and is predicted to be defective.
— **False Negative (FN):** The class is actually defective and is predicted to be non-defective.
— **True Negative (TN):** The class is actually non-defective and is predicted to be non-defective.
— **False Positive (FP):** The class is actually non-defective and is predicted to be defective.
— **Precision:** $\frac{TP}{TP+FP}$.
— **Recall:** $\frac{TP}{TP+FN}$.
— **F1-score:** $\frac{2*Precision*Recall}{Precision+Recall}$.
— Cohen's **Kappa:** A statistic that assesses the classifier's performance against random guessing [10]. $Kappa = \frac{Observed-Expected}{1-Expected}$ where
  – Observed: The proportionate agreement. $\frac{TP+TN}{TP+TN+FP+FN}$.
  – Expected: The probability of random agreement. $P_{Yes} + P_{No}$ where
    * P_{Yes}: Probability of positive agreement. $\frac{TP+FP}{TP+TN+FP+FN} * \frac{TP+FN}{TP+TN+FP+FN}$.
    * P_{No}: Probability of negative agreement. $\frac{TN+FP}{TP+TN+FP+FN} * \frac{TN+FN}{TP+TN+FP+FN}$.
— **AUC** [58] is the area under the curve, of true positives rate versus false positive rate, which is defined by setting multiple thresholds. AUC has the advantage to be threshold independent.
— **MCC** is commonly used in assessing the performance of classifiers dealing with unbalanced data [44], and is defined as $\frac{TP*TN-FP*FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$. Its interpretation is similar to correlation measures, i.e., $MCC < 0.2$ is considered to be low, $0.2 \leq MCC < 0.4$—fair, $0.4 \leq MCC < 0.6$—moderate, $0.6 \leq MCC < 0.8$—strong, and $MCC \geq 0.8$—very strong.

2.2.3 Measurement. Our measurement procedure consists of the following steps:

(1) *Labeling defective classes:* For each project, we created a dataset **D** by labeling classes in releases as defective or not. To do so, we used the "realistic approach" [79], i.e., using the affected version reported in Jira. This means that, given our point of observation (which, as explained below and shown as a camera photo in Figure 2, is where a training set is built), we consider the affected version from all bug reports opened at that time. for each bug, we consider as affected versions those indicated by in the bug report. Moreover, since the realistic approach is based on the use of the affected version that is reported in Jira, and since according to Table 4 a considerable number of defects do not report the affected version in Jira, then, using only the realistic approach would have produced several false-negative labels of classes. Therefore, to avoid noise in our datasets due to missing affected versions in JIRA, we used SZZ for defects where the affected version is not available. We re-implemented the SZZ algorithm [66] and improved it by ignoring comments,[2] indentation, white spaces, and documentation strings[3] as changes introducing defects. Thus, similarly to what we have done in a recent paper [73], we identify the introduction version of each defect using SZZ if the ticket was without affected version, or as the oldest of the affected versions in the ticket.

[2] https://goo.gl/X8fHFc.
[3] https://goo.gl/dNXb6N.

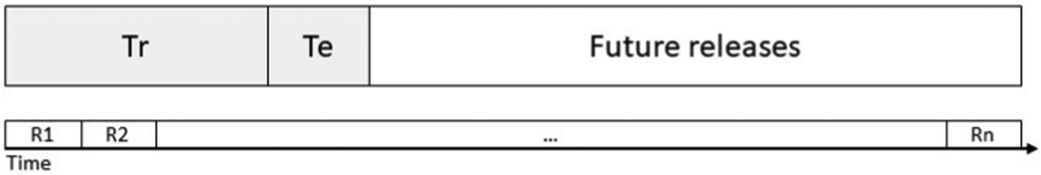


Fig. 1. The evaluation technique adopted in this article. The first 66% of data is used as the training set, and the remaining 33% as the test set. Several releases are unused to avoid that snoring impacts our results.

Afterward, regardless of how we identified the introduction version, we label the classes involved in the fix commit of the ticket as defective in releases in the range (introduction version, fixed version).

(2) *Avoiding snoring in our measurement:* Since in a recent paper [3] we proved that removing most recent releases implies removing snoring, then, for each project, we created SD by removing the last 50% of releases from D. Therefore, by using SD, rather than D, we are confident that our ground truth is not significantly affected by snoring.

(3) *Training and test sets:* Figure 1 describes our overall validation technique which is then customized for the two research questions. Our overall validation technique is a 66/33 holdout that preserves the order of data [15]. First, the data is ordered according to the releases of the project, i.e., we created D. Second, the last 50% of releases are removed to avoid that snoring impacts our ground truth by more than 1% [3], i.e., we created SD. Third, the first 66% of SD is used as a **training set** (**Tr**), and the remaining 33% of SD is used as a test set (**Te**). Figure 2 reports the validation technique used in this research question. In the top part of Figure 2, we have the ideal scenario: we create the test sets and training sets without snoring (i.e., TrNS and TeNS) by executing the labeling technique, which is represented with a photo camera, at the end of the project. On the bottom part of Figure 2, we replicate the realistic scenario: we create the training sets with snoring (i.e., TrS) by executing the labeling technique, which is represented with a photo camera, at data collection, i.e., at the end of the training set. For example, in project Bookkeeper, Tr is constructed using the first eight releases. The class BookieProtocol.java in releases 7 and 8 of TrS is marked as non-defective, but will become defective if observed after release 11 because of the defect BOOKKEEPER-1018.[4] Thus, the same exact class BookieProtocol.java in releases 7 and 8 is labeled as defective in TrNS and as non-defective in TrS.

(4) *Classifiers' performance:* We train each classifier on TrS vs. TrNS and tested on TeNS. Thus, we compare the performance of the same classifier when trained with snoring versus trained and without snoring.

We use 16 well-defined product and project metrics that previous work found to be useful for defect prediction [14, 17]. These predictor metrics are detailed in Table 5. We note that TrS and TrNS share all predictor metrics and values, and differ only in the defectiveness label of some classes, since classes are labeled at different points of time, in the training set.

Afterward, we perform four sub-steps.

(a) Feature Selection. We filter the predictor variables described above by using the correlation-based feature subset selection [25]. The approach evaluates the worth of a subset of attributes by considering the individual predictive capability of each feature, as well as the degree of redundancy between different features. The approach searches the space of attribute subsets by greedy hill-climbing augmented with a backtracking facility.

[4] https://issues.apache.org/jira/browse/BOOKKEEPER-1018.

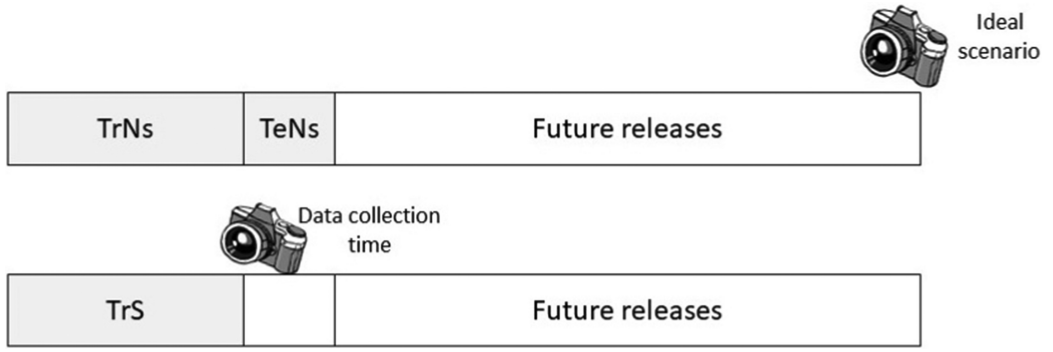


Fig. 2. A photo camera represents the technique that is used to label class defectiveness. We executed the technique at different points in time. On the bottom side, the labeling is performed in a realistic usage scenario, i.e., when one realistically collects the data to be used in the prediction model. This execution produces a training set with snoring (TrS). On the top side, the collection is performed in an ideal scenario, i.e., at the end of the project. This execution produces a training and test set without snoring (TrNS and TeNS).

Table 5. Defect Prediction Features

Metric	Description
Size	**Lines of code (LOC)**
LOC Touched	Sum over revisions of LOC added + deleted
NR	Number of revisions
Nfix	Number of bug fixes
Nauth	Number of authors
LOC Added	Sum over revisions of LOC added + deleted
MAX LOC Added	Maximum over revisions of LOC added
AVG LOC Added	Average LOC added per revision
Churn	Sum over revisions of added − deleted LOC
Max Churn	Maximum churn over revisions
Average Churn	Average churn over revisions
Change Set Size	Number of files committed together
Max Change Set	Maximum change set size over revisions
Average Change Set	Average change set size over revisions
Age	Age of Release
Weighted Age	Age of Release weighted by LOC touched

The approach starts with the empty set of attributes and performs a forward search by considering all possible single attribute additions and deletions at a given point.

(b) Classifiers. As classifiers we use the 14 used in a previous related paper [32]:
- Decision Stump: A single-level decision tree performing classification based on entropy [30].
- Decision Table: Two major parts: schema, the set of features included in the table, and a body, labeled instances defined by features in the schema. Given an unlabeled instance, try matching instance to record in the table [41].
- IBk: K-nearest neighbors classifier run with k = 1 [2].
- J48: Generates a pruned C4.5 decision tree [59].

— JRip: A propositional rule learner, **Repeated Incremental Pruning to Produce Error Reduction (RIPPER)** [11].
— KStar: Instance-based classifier using some similarity function. Uses an entropy-based distance function [9].
— Naive Bayes: Classifies records using estimator classes and applying Bayes theorem [31].
— Naive Bayes Updatable: An instance of the Naive Bayes classifier with different weight initial values and constraints [31].
— OneR: 1R classifier using the minimum-error attribute for prediction [29].
— PART: Uses separate-and-conquer, building partial C4.5 decision trees and turning the best leaf into a rule [19].
— Random Forest: Ensemble learning creating a collection of decision trees. Random trees correct for overfitting [7].
— REPTree: Fast decision tree learner. Builds a decision tree using information gain and variance, and prunes using reduced-error pruning [43].
— SMO: John Platt's sequential minimal optimization algorithm for training a support vector classifier [57].

(c) Tuning. Instead of tuning each single classifier, we used AutoWEKA [42], an automated approach for classifier selection and hyperparameter optimization. We run AutoWEKA for 2 hours on each training set (with or without snoring) of each project. Thus, we refer to *AutoWEKA* as the classifier and parameter selected by AutoWEKA for the specific training set.
(d) Test. We train each of the 14 classifiers, plus the AutoWEKA classifier, on TrS and test on TeNS and we train on TrNS, and we test on TeNS.

It is possible that snoring could depend on the distance between releases, measured in terms of days or commits. To verify whether RQ_1 results are affected by that, we compute the Spearman's rank correlation [67] between the number of days and commits per release and the model's relative loss. A high and statistically significant, positive or negative correlation, could indicate that results may depend on the distance between releases, i.e., snoring is more likely to occur for rapid releases (and therefore one may need to drop more releases) than for a longer distance between releases.

2.2.4 Analysis Procedure. We compare the accuracy of models when trained on a training set with vs. without snoring (i.e., TrS vs. TrNS) and tested on the test set without snoring (TeNS).

Since our data strongly deviate from normality, the hypotheses of this research question are tested using the Wilcoxon signed-rank test [76], and Cliff's delta (paired) effect size [23]. Also, since we repeat the statistical analysis multiple times (i.e., for each classifier), we adjust p-values using the Holm's correction procedure [28]. Such a procedure ranks n p-values in increasing order of value and multiplies the first one (i.e., the smaller) by n, the second by $n - 1$, and so on.

2.3 RQ_2: Does Defect Prediction Performance Improve if We Remove from the Training Set the Classes that are Labeled as Non-defective in the Last Releases?

We are in the shoes of a practitioner who reflects on whether training classifiers on less but non-noisy data improves the defect prediction performance over training classifiers on all data. At the time of the prediction, the practitioner does not know which specific class snores; the only information they can leverage to remove snoring classes is that (1) younger releases are more likely to contain snoring classes than older releases, and (2) classes labeled as not defective cannot be snoring. Thus, the only feasible and realistic approach that can be used to remove snoring, at the time of the prediction, is to remove all the classes labeled as defective in the last releases. However, this approach, since it is not perfectly accurate, is a double-edged sword; on the one

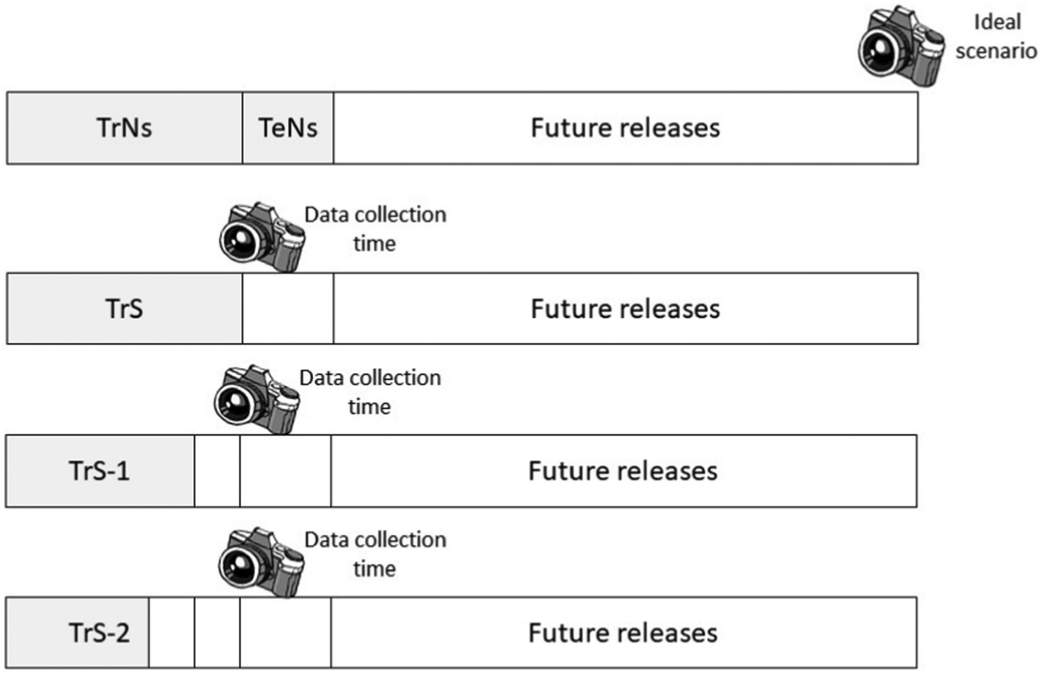


Fig. 3. The technique used to replicate the scenario where the user wants to use the defect prediction model and tries to improve its accuracy by removing classes labeled as not defective from the last 0, 1, 2, 3, and 4 releases of the training set with snoring.

hand, it removes all snoring data in the last releases; on the other hand, it removes data that are also not snoring (i.e., actually defective classes). To address this research question, we test the null hypothesis: H_{02}: there is no difference between the prediction performance achieved by classifiers when trained on an entire dataset versus a portion of the same dataset where classes labeled as non-defective are removed from last releases.

2.3.1 Independent Variables. The independent variable of RQ_2 is the number of releases from which classes labeled as non-defective are removed from the training set.

2.3.2 Dependent Variables. The dependent variable is the relative loss of the performance measures used to address RQ_2.

2.3.3 Measurement Procedure. Figure 3 reports the technique used to replicate the scenario where the user wants to use a defect prediction model and then tries to improve the prediction accuracy by removing from the training set the classes labeled as not defective in last releases. Thus, as in RQ1, the technique used to label class defectiveness is executed at the point in time where the model is used for prediction, i.e., at the end of the training set. Differently from RQ1, we remove from the training set the classes labeled as not defective in the last 0, 1, 2, 3, and 4 releases; we call these training sets TrS, TrS-1, TrS-2, TrS-3, and TrS-4, respectively. As in RQ1, the training and test sets without snoring (TrNS and TeNS) are created by executing the labeling technique at the end of the project.

For example, removing classes labeled as not defective from the last release of the training set of the *Bookkeeper* project, to form TrS-1, would eliminate classes that snore at that release such as `ZkLedgerUnderreplicationManager.java`.[5]

2.3.4 Analysis Procedure. We compare the accuracy of models when trained on training sets obtained by removing all classes labeled as not defective from the last 0, 1, 2, 3, and 4 releases and tested on a test set without snoring (TeNS).

Then, we test whether there is a correlation between the removal of non-defective classes of the last release(s) of the training set and the gain in terms of Precision, Recall, Cohen's kappa, MCC, and AUC. To this aim, we use a repeated-measure permutation test, using the Project as a random variable, and considering as an independent variable the number of removed releases and the choice of the classifier.

We also analyze the correlation between the gain in performance, provided by removing non-defective classes in the last release of the training set, and the frequency of releases in projects.

Finally, to promote replicability, we made available online all datasets, SZZ, and other developed scripts.[6]

3 EMPIRICAL STUDY RESULTS

In the following, we report the results of our study to address the research questions formulated in Section 1.

3.1 RQ_1: Does Snoring Reduce Defect Prediction Performance?

Table 6 reports, for each project, the number of classes that are labeled as Defective and Not Defective in TrNs, as Defective in TrS, and the loss as computed as the proportion of defective classes that are erroneously not labeled as defective in the TrS, (i.e., number of defective classes in TrNS − number of defective classes in TrS)/number of defective classes in TrNS). According to Table 6, for most of the projects, the majority of defective classes are affected by snoring (Loss > 50%). In the remainder of this research question, we observe how this loss affects defect prediction models.

Figure 4 reports, as boxplots, the distribution of Precision, Recall, F1-Score, AUC, MCC, and Cohen *k* for different classifiers and projects, achieved when trained on data with (red boxes) versus without (blue boxes) snoring. This same data is reported for each classifier in Figure 5. According to Figure 5, the presence of snoring in the training set decreases the performance in each of the 15 classifiers and in each of the six performance metrics. Figure 6 reports, as boxplots, the distribution of relative loss in Precision, Recall, AUC, F1-Score, Cohen *k*, and MCC for different classifiers and projects. According to Figure 6, the highest median relative loss, among performance metrics, is in Recall, and it is about 90%.

As explained in Section 2, the observed differences have been statistically analyzed using Wilcoxon signed-rank and Cliff's delta effect size (positive effect sizes indicate a difference in favor of results without snoring). As Table 7 shows, Recall is significantly higher without snoring, as confirmed in Figure 4, with an effect size that is always large. Instead, we found no significant differences in what concerns Precision.

Table 7 and Table 8 report the statistical test results about the differences in performances of defect prediction classifiers when trained on a dataset with versus without snoring. As Table 7 and Table 8 show, significant differences in terms of all performance metrics other than Precision

[5] https://tinyurl.com/ybupvq3q.
[6] http://doi.org/10.5281/zenodo.4394035.

Table 6. Snoring Classes: Proportion of Defective Classes Not Labeled as Defective in the Training Set

Project	TrNs		TrS	Loss
	Defective	Not Defective	Defective	
AVRO	54	1,249	17	31%
BOOKKEEPER	836	2,924	167	20%
CHUKWA	1,031	1,681	1,010	98%
CONNECTORS	784	3,963	690	88%
CRUNCH	298	1,460	152	51%
FALCON	199	368	193	97%
GIRAPH	592	2,991	154	26%
IVY	603	4,517	271	45%
OPENJPA	1,000	12,108	640	64%
PROTON	350	10,480	140	40%
SSHD	346	991	242	70%
STORM	2,208	5,220	1,700	77%
SYNCOPE	11,202	9,170	2,801	25%
TAJO	1,261	4,660	227	18%
TEZ	486	914	408	84%
TOMEE	326	5,914	293	90%
WHIRR	45	810	17	38%
ZEPPELIN	786	2,253	424	54%
ZOOKEEPER	434	1,693	308	71%

are observed for all classifiers other than decision_stump. Thus, we can reject H_10 and claim that **snoring significantly worsens the accuracy of all classifiers and all metrics other than Precision**.

RQ_1 Summary: The effect of snoring on classification performance is of large size on all classifiers and all metrics other than Precision.

3.2 RQ_2: Does Defect Prediction Performance Improve if We Remove from the Training Set the Classes that are Labeled as Non-defective in the Last Releases?

Figure 7 reports the distribution among datasets and classifiers, of different performance metrics (x-axis), when removing a specific number of last releases data (color). Table 9 summarizes Figure 7 by reporting the relative gain (%) on average among datasets and classifiers, by removing a specific number of releases of data on average across classifiers and datasets. According to Table 9:

(1) **Removing one release is better than removing no release in all performance metrics**.
(2) Removing three or four releases is worse than removing one or two releases in all performance metrics.
(3) Removing two releases is better than removing one release only in terms of Recall. In other words, removing one release is better than removing two releases in Precision and all the combined performance metrics.
(4) The gain in removing releases is particularly small in terms of AUC.
(5) Removing more than one release reduces Precision.

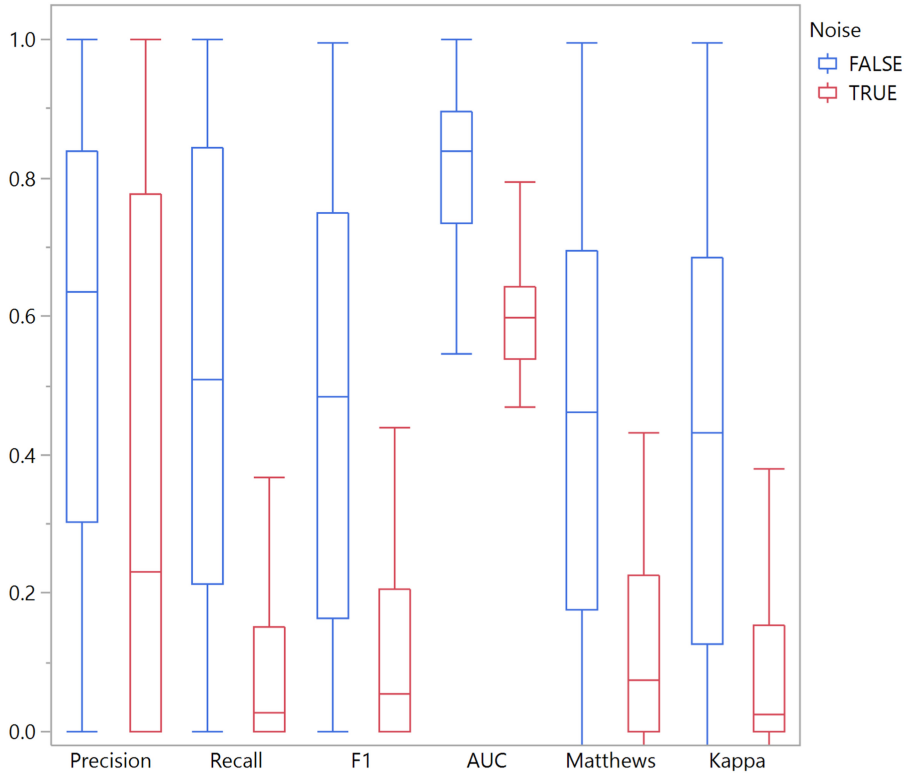


Fig. 4. Distribution of performance among classifiers and datasets achieved with versus without snoring.

Figure 8 reports the distribution of performance in each classifier, among projects, by removing, or not, non-defective classes in the last release of the training set. According to Figure 8, removing one release is better than removing no release of data for all classifiers and all performance metrics.

Table 10 reports the permutation test results on the difference in performance, in the cases of removing, or not, non-defective classes in the last release of the training set. According to Table 10, we can reject H_20 and claim that removing non-defective classes in the last release of the training set improves the performance of classifiers in all performance metrics other than Recall. Moreover, according to Table 10, the performance of classifiers varies across classifiers on all performance metrics.

Table 11 reports the Spearman correlation between the gain in performance, provided by removing non-defective classes in the last release of the training set, as measured by Precision and Recall, and the frequency of releases in projects, as computed in terms of the average number of days in a release and the average number of commits in a release. According to Table 11, there is a relevant and almost statistically significant inverse correlation between the gain in performance, provided by removing non-defective classes in the last release of the training set, and the number of commits per release. Thus, the smaller are the releases in terms of the number of commits, the higher is the gain in non-defective classes in the last release of the training set.

RQ_2 Summary: Using a training set that excludes non-defective classes from the last one or two releases increases the classifiers' performance in all six metrics. However, further exclusion worsens the models' performance.

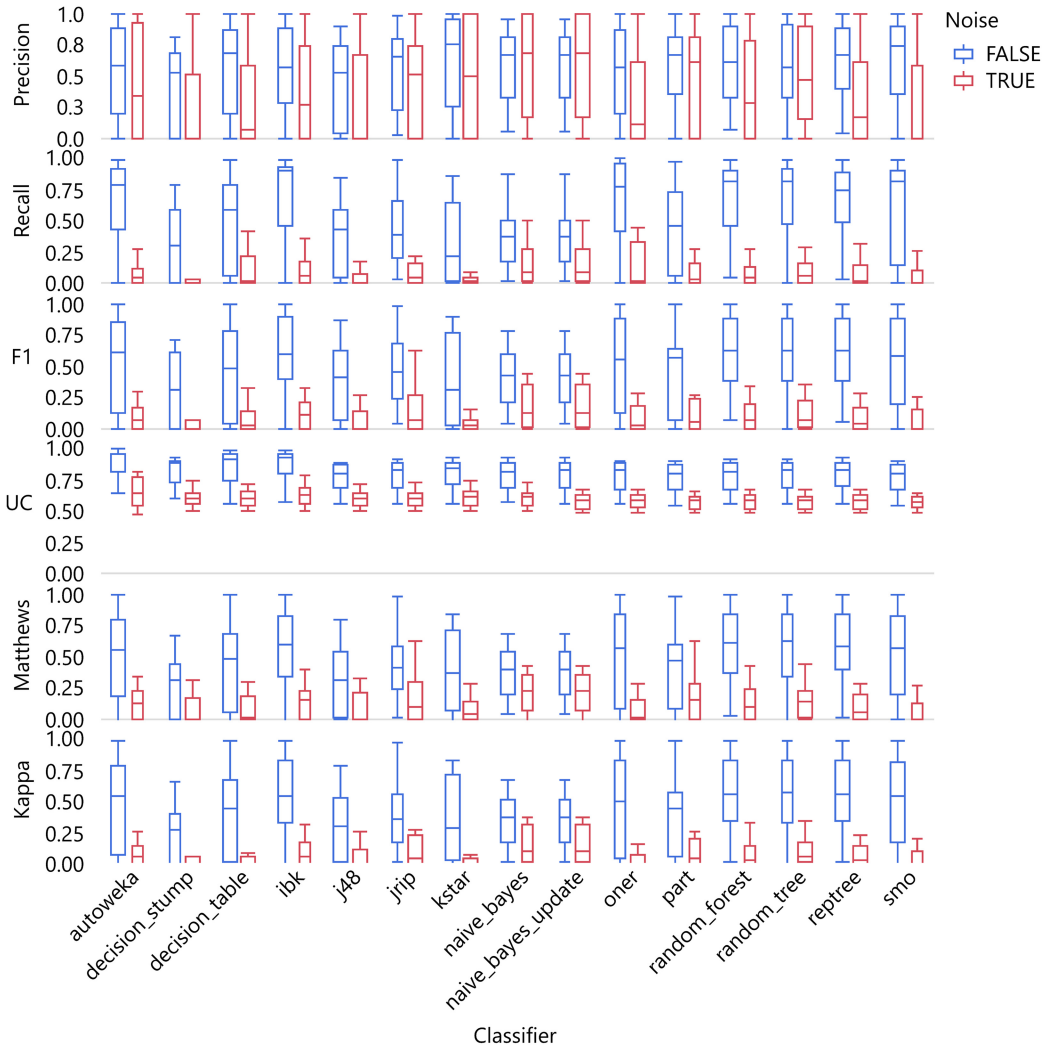


Fig. 5. Distribution of performance among datasets of different classifiers (x-axis) in the cases of with or without the snoring.

4 DISCUSSION

This section describes the implications of the existence of snoring on the performances of machine learning models to predict class defectiveness.

Based on RQ_1 results, the presence of snoring significantly decreases the classifiers' performance. For instance, the median Recall achieved by training a classifier on the entire dataset with snoring is 90% less than the Recall ideally achievable by training that same classifier on the same entire dataset without snoring.

It is interesting to note that we found no significant differences in what concerns Precision. One possible reason for this high impact of snoring on Recall and low impact on Precision is the fact that snoring likely increases the proportion of negatives in the training set. Thus, training

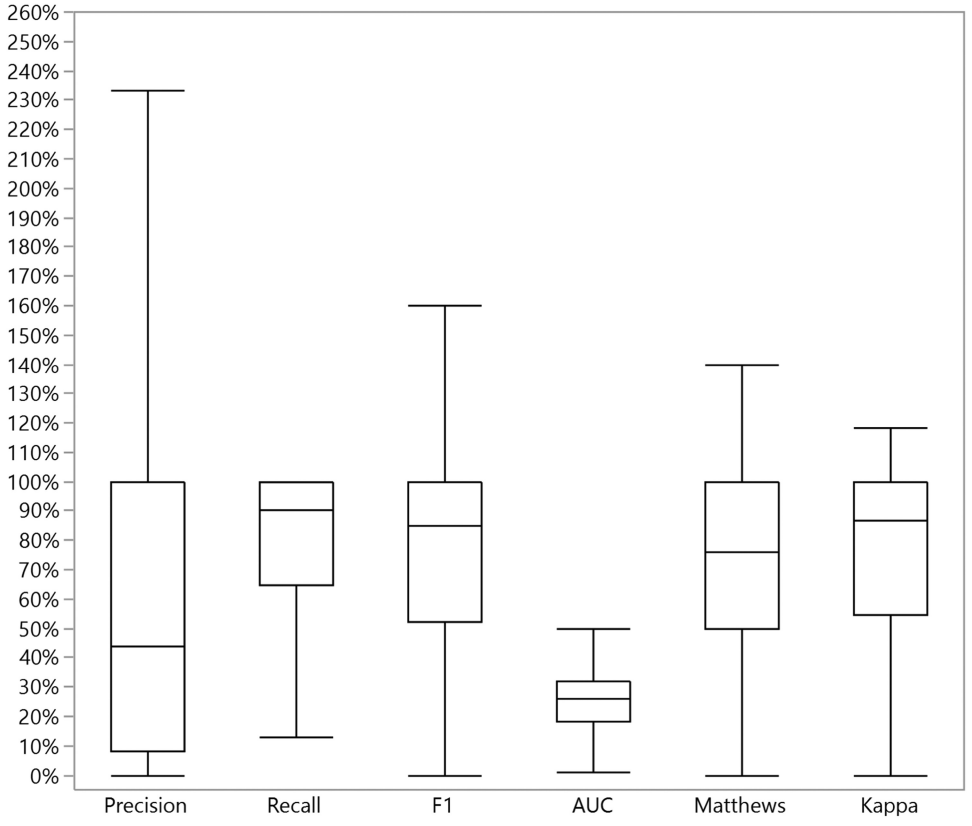


Fig. 6. Distribution of relative loss among classifiers and datasets.

classifiers providing a high number of negatives results in a high proportion of false negatives and hence a lower Recall.

Unfortunately, the snoring instances cannot be accurately identified yet, and therefore, an entire dataset without snoring is only ideal. What we know is that, by definition, the snoring classes are more frequent in last releases than in early releases. However, regardless of the number of releases we ignore, we know that we also ignore data that is valid, i.e., classes that are actually non-defective. The impossibility to remove only noise instances implies a tradeoff between the amount of data and the quality of the data used to feed prediction models.

Results from RQ_2 suggest that performances improve when removing non-defective data from the last release, and further improve on all metrics other than Precision when removing the last two releases. At the same time, going way too ahead, removing more than two releases, means training the model with less and older data, and then worsening the models' performances. Specifically, according to the RQ_2 results, removing the last two releases of data is better than removing only the last release in terms of Recall, but worse in terms of Precision. Therefore, based on the RQ_2 results, a good tradeoff is to not train classifiers with classes labeled as non-defective in the last release.

Researchers should identify which specific class in the last releases is erroneously labeled as non-defective. This advancement would allow a reduction in the number of removed non-defective classes and hence an improvement in the amount of data that can be used to feed prediction models.

Table 7. Impact of Snoring on Classifiers' Performance (RQ_1): Wilcoxon Signed-Rank Test Results and Cliff's Delta Effect Size on Precision and Recall for Different Machine Learning Classifiers

Classifier	Precision			Recall		
	p-value	*d*	Magnitude	*p*-value	*d*	Magnitude
autoweka	1.00	0.11	negligible	0.00	0.76	large
decision_stump	0.18	0.42	medium	0.01	0.58	large
decision_table	0.34	0.36	medium	0.01	0.64	large
ibk	0.07	0.38	medium	0.00	0.87	large
j48	0.34	0.30	small	0.01	0.60	large
jrip	1.00	0.22	small	0.00	0.76	large
kstar	1.00	0.21	small	0.01	0.58	large
naive_bayes	1.00	−0.05	negligible	0.00	0.65	large
naive_bayes_update	1.00	−0.05	negligible	0.00	0.65	large
oner	0.29	0.42	medium	0.00	0.79	large
part	1.00	0.17	small	0.00	0.64	large
random_forest	0.04	0.34	medium	0.00	0.90	large
random_tree	0.34	0.15	small	0.00	0.87	large
reptree	0.02	0.48	large	0.00	0.93	large
smo	0.01	0.55	large	0.00	0.76	large

Table 8. Impact of Snoring on Classifiers' Performance (RQ_1): Wilcoxon Signed-Rank Test Results and Cliff's Delta Effect Size on Cohen *k*, MCC, and AUC for Different Machine Learning Classifiers

Classifier	Cohen *k*			MCC			AUC		
	p-value	*d*	Magnitude	*p*-value	*d*	Magnitude	*p*-value	*d*	Magnitude
autoweka	0.00	0.62	large	0.00	0.64	large	0.00	0.84	large
decision_stump	0.02	0.45	medium	0.03	0.45	medium	0.00	0.87	large
decision_table	0.00	0.53	large	0.00	0.54	large	0.00	0.84	large
ibk	0.00	0.80	large	0.00	0.78	large	0.00	0.82	large
j48	0.01	0.52	large	0.02	0.51	large	0.00	0.82	large
jrip	0.02	0.60	large	0.03	0.58	large	0.00	0.82	large
kstar	0.00	0.58	large	0.00	0.54	large	0.00	0.82	large
naive_bayes	0.00	0.57	large	0.00	0.48	large	0.00	0.85	large
naive_bayes_update	0.00	0.57	large	0.00	0.48	large	0.00	0.84	large
oner	0.00	0.67	large	0.00	0.67	large	0.00	0.83	large
part	0.02	0.48	large	0.03	0.48	large	0.00	0.84	large
random_forest	0.00	0.80	large	0.00	0.78	large	0.00	0.84	large
random_tree	0.00	0.71	large	0.00	0.71	large	0.00	0.84	large
reptree	0.00	0.83	large	0.00	0.82	large	0.00	0.85	large
smo	0.00	0.75	large	0.00	0.75	large	0.00	0.84	large

Table 9. Relative Gain (%) in Performance Achieved by Removing a Specific Number of Releases of Data on Average Across Classifiers and Datasets

Removed Releases	Precision	Recall	F1	Kappa	Matthews	AUC
1	17%	41%	33%	39%	32%	3%
2	−1%	67%	37%	44%	31%	3%
3	−27%	45%	7%	−3%	−12%	2%
4	−39%	29%	9%	−18%	−25%	0%

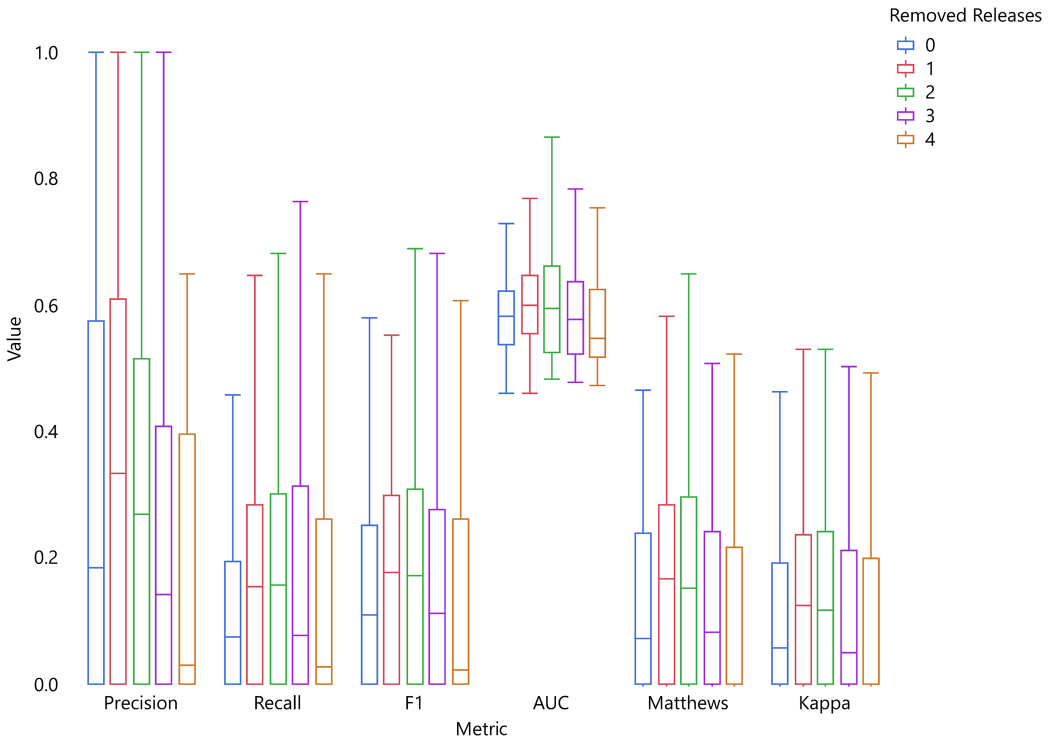


Fig. 7. Distribution of performance among datasets in the cases of removal of a specific number of last releases of data.

5 THREATS TO VALIDITY

In this section, we discuss the study threats to validity, and specifically construct, internal, conclusion, and external validity threats [78].

Construct validity threats are related to the relationship between theory and observation. The first construct validity threat is related to the dataset construction, i.e., how we created a dataset in which we know where snoring occurs. As explained in Section 2.2.3, we achieved this goal by removing the last 50% of the dataset. Although previous work suggests that this may drop about 99% of the snoring [3], we cannot exclude the presence of some snoring that we did not capture. To identify buggy classes, we combined the availability of the JIRA "affected version" with the use of the SZZ algorithm. The former provides a more realistic assessment of buggy classes; however, it is not always available, and for this reason, we combined it with SZZ. However, we are aware that SZZ determines the affected versions differently than what happens with the manually inserted label, i.e., considering all versions from the bug introduction until its fixing.

Another threat to construct validity is the absence of the use of any balancing technique. The impact of balancing on classifiers' accuracy has been analyzed in a context similar to ours such as **just-in-time** classifiers (**JIT**) [68]. As stressed for JIT, and known in other domains [77], balancing techniques generally improve Recall at the cost of reducing Precision with no impact on overall accuracy performance metrics. Table 12 reports the proportion of defective classes in the same training set when removing classes labeled as defective in the last [0–4] releases of the training set. According to Table 12, removing non-defective data from the last release resulted in a slightly more balanced dataset. Since the observed improvement is in terms of Precision, Recall, and AUC,

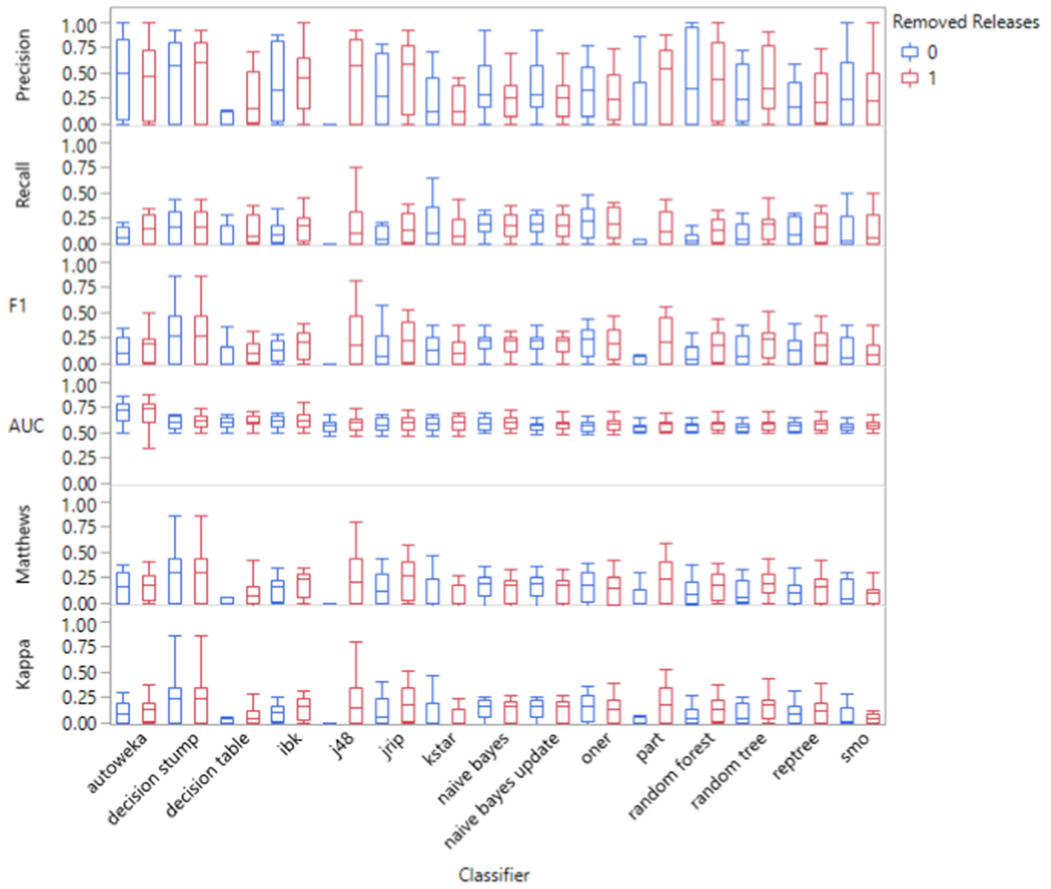


Fig. 8. Distribution of performance in each classifier among projects by removing, or not, non-defective classes in the last release of the training set.

then we are confident that the observed increment is due to less noise in the data rather than a more balanced dataset. It was surprising to observe an increment in Precision despite a more balanced dataset; i.e., the snoring-free data counterbalanced the negative effect on Precision of a more balanced dataset.

Internal validity threats concern variables internal to our study and not considered in our experiment that could influence our observations on the dependent variable. In our study, we split data into a 66% training set and a 33% test set. However, it is possible that different splits could produce varying results. As our results already show, the effect of dormant defects can vary based on the choice of a specific machine learning model used for defect prediction. We cannot exclude that other models not considered in our study could exhibit different performance variations when the dataset is affected by dormant defects and when not. Moreover, it is also possible that tuning the models' hyperparameters would produce different results as well. However, it is not our goal to find the best performing machine learning model for defect prediction, but to simply show that dormant defects have a significant effect on the models' performances. The article shows this is the case on all accuracy metrics other than recall.

Table 10. RQ_3: Permutation Test Results on the Effect of Dropping Releases on Classifier Performance Metrics

		Precision			
	Df	R Sum Sq	R Mean Sq	Iter	Pr(Prob)
Between projects effects:					
Classifier	1	0.13	0.13	51.00	0.80
Residuals	10	12.41	1.24		
Within project effects:					
Classifier	16	10.41	0.65	5,000.00	0.00
DropCount	1	2.20	2.20	5,000.00	0.00
Classifier:DropCount	16	1.79	0.11	5,000.00	0.02
Residuals	945	55.88	0.06		
		Recall			
	Df	R Sum Sq	R Mean Sq	Iter	Pr(Prob)
Between projects effects:					
Classifier	1	0.00	0.00	51.00	0.92
Residuals	10	7.40	0.74		
Within project effects:					
	Df	R Sum Sq	R Mean Sq	Iter	Pr(Prob)
Classifier	16	3.91	0.24	5,000.00	0.00
DropCount	1	0.09	0.09	5,000.00	0.01
Classifier:DropCount	16	1.02	0.06	5,000.00	0.00
Residuals	945	25.51	0.03		
		Kappa			
	Df	R Sum Sq	R Mean Sq	Iter	Pr(Prob)
Between projects effects:					
Classifier	1	0.21	0.21	169.00	0.37
Residuals	10	4.11	0.41		
Within project effects:					
DropCount	1	0.09	0.09	5,000.00	0.00
Classifier	16	2.01	0.13	5,000.00	0.00
DropCount:Classifier	16	0.36	0.02	5,000.00	0.20
Residuals	945	15.05	0.02		
		MCC			
	Df	R Sum Sq	R Mean Sq	Iter	Pr(Prob)
Between projects effects:					
Classifier	1	0.11	0.11	51.00	0.76
Residuals	10	4.70	0.47		
Within project effects:					
DropCount	1	0.22	0.22	5,000.00	0.00
Classifier	16	2.69	0.17	5,000.00	0.00
DropCount:Classifier	16	0.45	0.03	5,000.00	0.03
Residuals	945	18.26	0.02		
		AUC			
	Df	R Sum Sq	R Mean Sq	Iter	Pr(Prob)
Between projects effects:					
Classifier	1	0.00	0.00	51.00	0.98
Residuals	10	2.59	0.26		
Within project effects:					
DropCount	1	0.00	0.00	924.00	0.10
Classifier	16	0.67	0.04	5,000.00	0.00
DropCount:Classifier	16	0.00	0.00	483.00	1.00
Residuals	945	3.05	0.00		

Table 11. Spearman Correlation between the Gain in Performance, Provided by Removing Non-Defective Classes in the Last Release of the Training Set, and the Frequency of Releases in Projects

Frequency	Metric	Rho	Pvalue
Days per release	Precision	−0.0979	0.7621
Commits per release	Precision	−0.5734	0.0513
Days per release	Recall	0.2098	0.5128
Commits per release	Recall	0.0839	0.7954

Table 12. Proportion of Defective Classes in the Same Training Set When Removing Classes Labeled as Defective in the Last 0, 1, 2, 3, and 4 Releases of the Training Set

Project Name	Defective - 0	Defective - 1	Defective - 2	Defective - 3	Defective - 4
AVRO	0.03	0.04	0.05	0.08	0.12
BOOKKEEPER	0.19	0.23	0.3	0.42	0.62
CHUKWA	0	0	0	0	0
CONNECTORS	0.02	0.03	0.03	0.04	0.07
CRUNCH	0.09	0.12	0.18	0.31	0.76
FALCON	0.01	0.01	0.02	1	1
GIRAPH	0.15	0.22	0.29	0.42	0.76
IVY	0.08	0.08	0.1	0.11	0.13
OPENJPA	0.04	0.05	0.06	0.1	0.16
PROTON	0.02	0.02	0.02	0.02	0.02
SSHD	0.1	0.13	0.16	0.21	0.3
STORM	0.09	0.11	0.12	0.14	0.17
SYNCOPE	1.16	1.16	1.32	1.54	1.67
TAJO	0.23	0.33	0.5	0.78	1.17
TEZ	0.08	0.18	1	1	1
THRIFT	0.14	0.17	0.21	0.27	0.37
WHIRR	0.04	0.06	0.1	0.25	1
ZEPPELIN	0.17	0.22	0.3	0.39	0.57
ZOOKEEPER	0.05	0.07	0.08	0.11	0.17

Conclusion validity threats regard issues that affect the ability to draw accurate conclusions about relations between the treatments and the outcome of an experiment [78]. We see no major threat to conclusion validity.

External validity threats concern the generalizability of our results, and are especially related to our choice and the representativeness of the artifacts (projects) considered in the study. This study used 19 datasets and hence could be deemed of low generalization compared to studies using hundreds of datasets. However, our datasets are large if we considered the number of releases (616) and defects (4101) and as stated by Nagappan et al. [51], "*more is not necessarily better*." Finally, we preferred to test our hypotheses only on datasets in which we were confident quality is high and that are close to industry.

6 RELATED WORK AND BACKGROUND

In recent and past years, several studies investigated approaches for creating defect prediction datasets [12, 16, 39, 53, 62, 66] such as SZZ and others. Other studies investigated how to select these datasets [18, 22, 51, 63].

Several authors have discussed factors hindering the accuracy and evaluation of defect prediction models [1, 5, 20, 46, 47, 70, 72, 80]. With respect to the aforementioned work, we empirically show that snoring constitutes an additional factor hindering their accuracy.

Chen et al. [8] firstly introduced the concept of *dormant defects* (though called dormant bugs). They showed that dormant defects are fixed faster and by more experienced developers. Similarly, Rodriguez-Perez et al. [61] and da Costa et al. [13] show that the time to fix a defect, i.e., the sleeping phenomenon, is on average about 1 year. Thus, we conclude that dataset creation will miss most defects on releases that are less than a year old. In a recent paper [3], we reported that many defects are dormant and many classes snore. Specifically, on average among projects, most of the defects in a project are dormant for more than 20% of the existing releases, and in the majority of the projects the missing rate is more than 25% even if we remove the last 50% of releases. Concerning previous work on dormant defects, including ours, in this article, we quantify, for the first time, the effect of dormant defects on defect prediction accuracy, their evaluation, and we provide and evaluate a countermeasure.

A problem of paramount importance when creating defect prediction models is represented by the presence of noise in the underlying datasets. In this context, different works study the sources of noise and foresee countermeasures for that. Kim et al. [36] measured the impact of noise on defect prediction models and provided guidelines for acceptable noise levels. They also propose a noise detection and elimination algorithm to address this problem. However, the noise studied and removed is supposed to be random. Tantithamthavorn et al. [69] found that (1) issue report mislabeling is not random; (2) precision is rarely impacted by mislabeled issue reports, suggesting that practitioners can rely on the accuracy of modules labeled as defective by models that are trained using noisy data; however, (3) models trained on noisy data typically achieve about 60% of the recall of models trained on clean data. Herzig et al. [27] reported that 39% of files marked as defective never had a defect actually. They discuss the impact of this misclassification on earlier studies and recommend manual data validation for future studies. Rahman et al. [60] showed that size always matters as much as bias direction, and in fact, much more than bias direction when considering information-retrieval measures such as AUCROC and F-score. This result indicates that at least for prediction models, even when dealing with sampling bias, simply finding larger samples can sometimes be sufficient. Bird et al. [6] found that bias is a critical problem that threatens both the effectiveness of processes that rely on biased datasets to build prediction models and the generalizability of hypotheses tested on biased data. Similarly to previous work, we study a source of noise in defect prediction datasets, i.e., snoring.

McIntosh and Kamei [45] recently found that in the context of JIT prediction, training prediction models on the last 3 months of data provides more accurate results than training on more past data. Their results seem to be in contrast to ours; we observed more accurate results by ignoring recent data; they observed more accurate results by ignoring old data. There are several possible reasons for this inconsistency. First, the different contexts; we classify classes whereas they classify commit. Second, the analyzed projects: they analyze two projects that are different from our 19 projects. Third, the two results might be consistent; it could be that the optimal data to train the models is the one in the middle, i.e., not too old (as shown in McIntosh and Kamei [45]), not too recent (as shown here). Thus, data for prediction models might act as vegetables, and you do not want them either unripe or rotten.

7 CONCLUSION

Dormant defects refer to defects discovered several releases after their introduction [8, 61]. While this concept is well-known in the software engineering community, its impact on defect prediction models was not investigated before.

In this article, we empirically analyze the effect of dormant defects in causing classes to be tagged as non-defective in a defect prediction dataset and therefore affect the performance of defect prediction models. We analyze data from more than 4,000 defects and 600 releases of 19 open source projects from the Apache ecosystem and 15 machine learning defect prediction classifiers. Our results show that (i) the presence of snoring decreases the recall of defect prediction classifiers, while not significantly affecting precision; and (ii) removing data from recent releases significantly improves the classifiers' performance. At the same time, going too behind worsens performances. In summary, this article provides insights on how to create a software defect dataset by mitigating the effect of snoring on classifiers performances.

While, in principle, using recent data could be valuable when building a defect prediction dataset (i.e., the system characteristics are more similar to the current status), this article showed how recent releases may contain defects that have not been discovered yet. For such a reason, classes with only such defects become "dormant" and represent false cases of defect-free classes in the dataset. Without doing better analysis and testing there is no simple way to discover defects in recent versions. At the same time, this article empirically shows the extent to which relying on less recent releases reduces the snoring effect in defect prediction results.

In terms of future work, we plan to extend this research as follows:

(1) *Combining noise removal techniques:* There are several types of noise currently known in defects datasets, including snoring and misclassification [27, 69]. However, no approach can remove all noises from a dataset.
(2) *Considering distribution as inputs for the defect prediction models:* We envision a model that accepts, as an input, a distribution of values, rather than a point-value [64]. In other words, since we cannot be sure about the defectiveness of a class in the training set, then, intuitively, the defectiveness of a class should be modeled by a confidence interval rather than a binary measure.
(3) *Analyzing the impact of snoring on existing research on factors affecting the performance of defect prediction:* previous research has shown the impact of tuning [1, 20] or balancing [68] on defect prediction. Such an impact could be influenced by snoring, and it would therefore be interesting to investigate that effect.
(4) *Application in the context of JIT Defect Prediction:* JIT prediction models have become sufficiently robust so that they are now incorporated into the development cycle of some companies [45]. We can replicate the present study on JIT models to understand the effect snoring has on the performance of these models.

REFERENCES

[1] Amritanshu Agrawal and Tim Menzies. 2018. Is "better data" better than "better data miners"?: On the benefits of tuning SMOTE for defect prediction. In *Proceedings of the 40th International Conference on Software Engineering (ICSE'18)*. 1050–1061.
[2] David Aha and Dennis Kibler. 1991. Instance-based learning algorithms. *Machine Learning* 6 (1991), 37–66.
[3] Aalok Ahluwalia, Davide Falessi, and Massimiliano Di Penta. 2019. Snoring: A noise in defect prediction datasets. In *Proceedings of the 16th International Conference on Mining Software Repositories (MSR'19)*. IEEE Press, Piscataway, NJ, 63–67. https://doi.org/10.1109/MSR.2019.00019
[4] Victor R. Basili, Lionel C. Briand, and Walcélio L. Melo. 1996. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on Software Engineering* 22, 10 (1996), 751–761.
[5] Sean Bayley and Davide Falessi. 2018. Optimizing prediction intervals by tuning random forest via meta-validation. CoRR abs/1801.07194. arXiv:1801.07194. http://arxiv.org/abs/1801.07194.
[6] Christian Bird, Adrian Bachmann, Eirik Aune, John Duffy, Abraham Bernstein, Vladimir Filkov, and Premkumar Devanbu. 2009. Fair and balanced?: Bias in bug-fix datasets. In *Proceedings of the the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE'09)*. ACM, New York, NY, 121–130. https://doi.org/10.1145/1595696.1595716

[7] Leo Breiman. 2001. Random forests. *Machine Learning* 45, 1 (2001), 5–32.
[8] Tse-Hsun Chen, Meiyappan Nagappan, Emad Shihab, and Ahmed E. Hassan. 2014. An empirical study of dormant bugs. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR'14)*. ACM, New York, NY, 82–91. https://doi.org/10.1145/2597073.2597108
[9] John G. Cleary and Leonard E. Trigg. 1995. K*: An instance-based learner using an entropic distance measure. In *12th International Conference on Machine Learning*. 108–114.
[10] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46.
[11] William W. Cohen. 1995. Fast effective rule induction. In *12th International Conference on Machine Learning*. Morgan Kaufmann, 115–123.
[12] Daniel Alencar Da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2016. A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes. *IEEE Transactions on Software Engineering* 43, 7 (2016), 641–657.
[13] Daniel Alencar da Costa, Shane McIntosh, Weiyi Shang, Uirá Kulesza, Roberta Coelho, and Ahmed E. Hassan. 2017. A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes. *IEEE Transactions on Software Engineering* 43, 7 (2017), 641–657.
[14] Marco D'Ambros, Michele Lanza, and Romain Robbes. 2012. Evaluating defect prediction approaches: A benchmark and an extensive comparison. *Empirical Software Engineering*. 17, 4-5 (Aug. 2012), 531–577. https://doi.org/10.1007/s10664-011-9173-9
[15] Davide Falessi, Jacky Huang, Likhita Narayana, Jennifer Thai, and Burak Turhan. 2020. On the need of preserving order of data when validating within-project defect classifiers. *Empirical Software Engineering* 25 (2020), 4805–4830. https://doi.org/10.1007/s10664-020-09868-x
[16] Davide Falessi and Max Jason Moede. 2018. Facilitating feasibility analysis: The pilot defects prediction dataset maker. In *Proceedings of the 4th ACM SIGSOFT International Workshop on Software Analytics*. 15–18.
[17] Davide Falessi, Barbara Russo, and Kathleen Mullen. 2017. What if I had no smells?. In *Proceedings of the 11th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM'17)*. IEEE Press, Piscataway, NJ, 78–84. https://doi.org/10.1109/ESEM.2017.14
[18] Davide Falessi, Wyatt Smith, and Alexander Serebrenik. 2017. STRESS: A semi-automated, fully replicable approach for project selection. In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, (ESEM'17)* . 151–156. https://doi.org/10.1109/ESEM.2017.22
[19] Eibe Frank and Ian H. Witten. 1998. Generating accurate rule sets without global optimization. In *15th International Conference on Machine Learning*, J. Shavlik (Ed.). Morgan Kaufmann, 144–151.
[20] Wei Fu, Tim Menzies, and Xipeng Shen. 2016. Tuning for software analytics: Is it really necessary? *Information & Software Technology* 76 (2016), 135–146.
[21] Kehan Gao, Taghi M. Khoshgoftaar, Huanjing Wang, and Naeem Seliya. 2011. Choosing software metrics for defect prediction: An investigation on feature selection techniques. *Software: Practice and Experience* 41, 5 (2011), 579–606.
[22] Georgios Gousios and Diomidis Spinellis. 2014. Conducting quantitative software engineering studies with alitheia core. *Empirical Software Engineering* 19, 4 (2014), 885–925.
[23] Robert J. Grissom and John J. Kim. 2005. *Effect Sizes for Research: A Broad Practical Approach* (2nd ed.). Lawrence Earlbaum Associates.
[24] Tibor Gyimóthy, Rudolf Ferenc, and István Siket. 2005. Empirical validation of object-oriented metrics on open source software for fault prediction. *IEEE Transactions on Software Engineering* 31, 10 (2005), 897–910.
[25] Mark A. Hall. 1998. *Correlation-Based Feature Subset Selection for Machine Learning*. Ph.D. Dissertation. University of Waikato, Hamilton, New Zealand.
[26] Ahmed E. Hassan. 2009. Predicting faults using the complexity of code changes. In *Proceedings of the 31st International Conference on Software Engineering (ICSE'09)*. IEEE, 78–88. https://doi.org/10.1109/ICSE.2009.5070510
[27] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It's not a bug, it's a feature: How misclassification impacts bug prediction. In *Proceedings of the 2013 International Conference on Software Engineering (ICSE'13)*. IEEE Press, Piscataway, NJ, 392–401. http://dl.acm.org/citation.cfm?id=2486788.2486840.
[28] Sture Holm. 1979. A simple sequentially rejective Bonferroni test procedure. *Scandinavian Journal on Statistics* (1979).
[29] Robert C. Holte. 1993. Very simple classification rules perform well on most commonly used datasets. *Machine Learning* 11 (1993), 63–91.
[30] Wayne Iba and Pat Langley. 1992. Induction of one-level decision trees. *Machine Learning Proceedings 1992* (1992), 233–240. https://doi.org/10.1016/b978-1-55860-247-2.50035-8
[31] George H. John and Pat Langley. 1995. Estimating continuous distributions in Bayesian classifiers. In *11th Conference on Uncertainty in Artificial Intelligence*. Morgan Kaufmann, 338–345.

[32] Leif Jonsson, Markus Borg, David Broman, Kristian Sandahl, Sigrid Eldh, and Per Runeson. 2016. Automated bug assignment: Ensemble-based machine learning in large scale industrial contexts. *Empirical Software Engineering* 21, 4 (2016), 1533–1578. https://doi.org/10.1007/s10664-015-9401-9

[33] Yasutaka Kamei, Takafumi Fukushima, Shane McIntosh, Kazuhiro Yamashita, Naoyasu Ubayashi, and Ahmed E. Hassan. 2016. Studying just-in-time defect prediction using cross-project models. *Empirical Software Engineering* 21, 5 (2016), 2072–2106.

[34] Taghi M. Khoshgoftaar, Edward B. Allen, Nishith Goel, Amit Nandi, and John McMullan. 1996. Detection of software modules with high debug code churn in a very large legacy system. In *7th International Symposium on Software Reliability Engineering (ISSRE'96)*. IEEE Computer Society, 364–371. https://doi.org/10.1109/ISSRE.1996.558896

[35] Sunghun Kim, E. James Whitehead, Jr., and Yi Zhang. 2008. Classifying software changes: Clean or buggy? *IEEE Transactions on Software Engineering* 34, 2 (2008), 181–196. https://doi.org/10.1109/TSE.2007.70773

[36] Sunghun Kim, Hongyu Zhang, Rongxin Wu, and Liang Gong. 2011. Dealing with noise in defect prediction. In *Proceedings of the 33rd International Conference on Software Engineering (ICSE'11)*. ACM, New York, NY, 481–490.

[37] Sunghun Kim, Thomas Zimmermann, E. James Whitehead, Jr., and Andreas Zeller. 2007. Predicting faults from cached history. In *29th International Conference on Software Engineering (ICSE'07)*. 489–498.

[38] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Whitehead, Jr. 2006. Automatic identification of bug-introducing changes. In *21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*. 81–90.

[39] Sunghun Kim, Thomas Zimmermann, Kai Pan, and E. James Whitehead, Jr.. 2006. Automatic identification of bug-introducing changes. In *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE'06)*. IEEE Computer Society, 81–90. https://doi.org/10.1109/ASE.2006.23

[40] Pavneet Singh Kochhar, Yuan Tian, and David Lo. 2014. Potential biases in bug localization: Do they matter?. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE'14)*. ACM, New York, NY, 803–814.

[41] Ron Kohavi. 1995. The power of decision tables. In *8th European Conference on Machine Learning*. Springer, 174–189.

[42] Lars Kotthoff, Chris Thornton, Holger H. Hoos, Frank Hutter, and Kevin Leyton-Brown. 2017. Auto-WEKA 2.0: Automatic model selection and hyperparameter optimization in WEKA. *Journal of Machine Learning Research* 18, 25 (2017), 1–5. http://jmlr.org/papers/v18/16-261.html.

[43] Yishay Mansour. 1997. Pessimistic decision tree pruning based on tree size. In *14th International Conference on Machine Learning*. 195–201.

[44] B. W. Matthews. 1975. Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica Et Biophysica Acta (BBA)* 2, 405 (1975), 442–451.

[45] Shane McIntosh and Yasutaka Kamei. 2018. Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction. *IEEE Transactions on Software Engineering* 44, 5 (2018), 412–428.

[46] Tim Menzies, Andrew Butcher, David R. Cok, Andrian Marcus, Lucas Layman, Forrest Shull, Burak Turhan, and Thomas Zimmermann. 2013. Local versus global lessons for defect prediction and effort estimation. *IEEE Transactions on Software Engineering* 39, 6 (2013), 822–834.

[47] Tim Menzies, Zach Milton, Burak Turhan, Bojan Cukic, Yue Jiang, and Ayse Basar Bener. 2010. Defect prediction from static code features: Current results, limitations, new approaches. *Automated Software Engineering* 17, 4 (2010), 375–407.

[48] Raimund Moser, Witold Pedrycz, and Giancarlo Succi. 2008. A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction. In *30th International Conference on Software Engineering (ICSE'08)*. 181–190.

[49] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. 2017. Curating github for engineered software projects. *Empirical Software Engineering* 22, 6 (Dec. 2017), 3219–3253. https://doi.org/10.1007/s10664-017-9512-6

[50] Meiyappan Nagappan, Thomas Zimmermann, and Christian Bird. 2013. Diversity in software engineering research. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE'13)*. ACM, New York, NY, 466–476.

[51] Meiyappan Nagappan, Thomas Zimmermann, and Christian Bird. 2013. Diversity in software engineering research. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering*. ACM, 466–476.

[52] Nachiappan Nagappan and Thomas Ball. 2005. Use of relative code churn measures to predict system defect density. In *27th International Conference on Software Engineering (ICSE'05)*, Gruia-Catalin Roman, William G. Griswold, and Bashar Nuseibeh (Eds.). ACM, 284–292. https://doi.org/10.1145/1062455.1062514

[53] Edmilson Campos Neto, Daniel Alencar da Costa, and Uirá Kulesza. 2018. The impact of refactoring changes on the SZZ algorithm: An empirical study. In *25th International Conference on Software Analysis, Evolution and Reengineering (SANER'18)*. 380–390. https://doi.org/10.1109/SANER.2018.8330225

[54] Niclas Ohlsson and Hans Alberg. 1996. Predicting fault-prone software modules in telephone switches. *IEEE Transactions on Software Engineering* 22, 12 (1996), 886–894. https://doi.org/10.1109/32.553637

[55] Thomas J. Ostrand and Elaine J. Weyuker. 2004. A tool for mining defect-tracking systems to predict fault-prone files. In *Proceedings of the 1st International Workshop on Mining Software Repositories (MSR@ICSE'04)*, Ahmed E. Hassan, Richard C. Holt, and Audris Mockus (Eds.). 85–89.

[56] Thomas J. Ostrand, Elaine J. Weyuker, and Robert M. Bell. 2005. Predicting the location and number of faults in large software systems. *IEEE Transactions on Software Engineering* 31, 4 (2005), 340–355.

[57] John Platt. 1998. Fast training of support vector machines using sequential minimal optimization. In *Advances in Kernel Methods - Support Vector Learning*, B. Schoelkopf, C. Burges, and A. Smola (Eds.). MIT Press. http://research.microsoft.com/~jplatt/smo.html.

[58] David M. W. Powers. 2007. Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technology* 2, 1 (2007), 37–63.

[59] Ross Quinlan. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann Publishers, San Mateo, CA.

[60] Foyzur Rahman, Daryl Posnett, Israel Herraiz, and Premkumar Devanbu. 2013. Sample size vs. bias in defect prediction. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE'13)*. ACM, New York, NY, 147–157.

[61] Gema Rodriguez-Perez, Gregorio Robles, and Jesus M. Gonzalez-Barahona. 2017. How much time did it take to notify a bug? Two case studies: ElasticSearch and Nova. In *Proceedings of the 8th Workshop on Emerging Trends in Software Metrics (WETSoM'17)*. IEEE Press, Piscataway, NJ, 29–35. https://doi.org/10.1109/WETSoM.2017..6

[62] Gema Rodríguez-Pérez, Andy Zaidman, Alexander Serebrenik, Gregorio Robles, and Jesús M. González-Barahona. 2018. What if a bug has a different origin?: Making sense of bugs without an explicit bug introducing change. In *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM'18)*. ACM, New York, NY, Article 52, 4 pages. https://doi.org/10.1145/3239235.3267436

[63] Daniel Rozenberg, Ivan Beschastnikh, Fabian Kosmale, Valerie Poser, Heiko Becker, Marc Palyart, and Gail C. Murphy. 2016. Comparing repositories visually with repograms. In *Proceedings of the 13th International Conference on Mining Software Repositories (MSR'16)*. 109–120.

[64] Mehrdad Sabetzadeh, Davide Falessi, Lionel Briand, and Stefano Di Alesio. 2013. A goal-based approach for qualification of new technologies: Foundations, tool support, and industrial validation. *Reliability Engineering and System Safety* 119 (2013), 52–66.

[65] Martin Shepperd, Qinbao Song, Zhongbin Sun, and Carolyn Mair. 2013. Data quality: Some comments on the NASA software defect datasets. *IEEE Transactions on Software Engineering* 39, 9 (Sept. 2013), 1208–1215. https://doi.org/10.1109/TSE.2013.11

[66] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When do changes induce fixes?. In *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR'05)*. ACM, New York, NY, 1–5. https://doi.org/10.1145/1082983.1083147

[67] Charles Spearman. 1904. The proof and measurement of association between two things. *American Journal of Psychology* 15, 1 (1904), 72–101.

[68] Chakkrit Tantithamthavorn, Ahmed E. Hassan, and Kenichi Matsumoto. 2018. The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. https://arxiv.org/abs/cs.SE/1801.10269.

[69] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, Akinori Ihara, and Kenichi Matsumoto. 2015. The impact of mislabelling on the performance and interpretation of defect prediction models. In *Proceedings of the 37th International Conference on Software Engineering - Volume 1 (ICSE'15)*. IEEE Press, Piscataway, NJ, 812–823. http://dl.acm.org/citation.cfm?id=2818754.2818852.

[70] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2016. Automated parameter optimization of classification techniques for defect prediction models. In *Proceedings of the 38th International Conference on Software Engineering (ICSE'16)* . 321–332.

[71] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, and Kenichi Matsumoto. 2018. The impact of automated parameter optimization on defect prediction models. CoRR abs/1801.10270. arXiv:1801.10270. http://arxiv.org/abs/1801.10270.

[72] Burak Turhan, Tim Menzies, Ayse Basar Bener, and Justin S. Di Stefano. 2009. On the relative value of cross-company and within-company data for defect prediction. *Empirical Software Engineering* 14, 5 (2009), 540–578.

[73] Bailey Vandehei, Daniel Alencar da Costa, and Davide Falessi. 2020. Leveraging the defects life cycle to label affected versions and defective classes, accepted for publication at TOSEM. CoRR abs/2011.05531. arXiv:2011.05531. https://arxiv.org/abs/2011.05531.

[74] Song Wang, Taiyue Liu, and Lin Tan. 2016. Automatically learning semantic features for defect prediction. In *Proceedings of the 38th International Conference on Software Engineering (ICSE'16)*. 297–308.

[75] Elaine J. Weyuker, Thomas J. Ostrand, and Robert M. Bell. 2010. Comparing the effectiveness of several modeling methods for fault prediction. *Empirical Software Engineering* 15, 3 (2010), 277–295. https://doi.org/10.1007/s10664-009-9111-2

[76] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics Bulletin* 1, 6 (1945), 80–83. http://www.jstor.org/stable/3001968.

[77] Ian H. Witten, Eibe Frank, and Mark A. Hall. 2011. *Data Mining: Practical Machine Learning Tools and Techniques* (3rd ed.). Morgan Kaufmann Publishers Inc., San Francisco, CA.

[78] Claes Wohlin, Per Runeson, Martin Hst, Magnus C. Ohlsson, Bjrn Regnell, and Anders Wessln. 2012. *Experimentation in Software Engineering*. Springer Publishing Company, Inc.

[79] Suraj Yatish, Jirayus Jiarpakdee, Patanamon Thongtanunam, and Chakkrit Tantithamthavorn. 2019. Mining software defects: Should we consider affected releases? In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE'19)*, Tevfik Bultan and Jon Whittle (Eds.). 654–665. https://2019.icse-conferences.org/home, https://2019.icse-conferences.org/.

[80] Thomas Zimmermann, Nachiappan Nagappan, Harald C. Gall, Emanuel Giger, and Brendan Murphy. 2009. Cross-project defect prediction: A large scale experiment on data vs. Domain vs. Process. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT International Symposium on Foundations of Software Engineering, 2009*. 91–100.

[81] Thomas Zimmermann, Rahul Premraj, and Andreas Zeller. 2007. Predicting defects for eclipse. In *Proceedings of the 3rd International Workshop on Predictor Models in Software Engineering (PROMISE'07)*. IEEE Computer Society, Washington, DC, 9.

Received September 2020; revised December 2020; accepted May 2021